

Core Empirical Risk

$$R(w) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i), \quad \min_w R(w)$$

Gradient descent: $w_{t+1} = w_t - \eta \nabla_w R(w_t)$.

Things to know

$$\nabla_b(a^T b) = a \quad \nabla_a \|a\|^2 = 2a \quad \|a\|^2 = a^T a$$
$$\sigma(z) = \frac{1}{1+e^{-z}}$$

1 Linear Regression

Model

$$\hat{y}_i = w^T x_i, \quad R(w) = \frac{1}{N} \sum_i (w^T x_i - y_i)^2$$
$$\nabla_w R(w) = \frac{2}{N} \sum_i (w^T x_i - y_i) x_i$$
$$X^T X w^* = X^T y, \quad w^* = (X^T X)^{-1} X^T y = X^\dagger y$$

Fails if: $N < D \Rightarrow \text{rank}(X) < D$.
Multi-output: $W^* = X^\dagger Y$.

For classification

$$\min_W \sum_i \|W^T x_i - y_i\|^2, \quad W^* = X^\dagger Y$$
$$\hat{y}_i = W^{*T} x_i, \quad \hat{c} = \arg \max_j \hat{y}_i^{(j)}$$

Regression Metrics

MSE: $\frac{1}{N} \sum (y_i - \hat{y}_i)^2$ RMSE: $\sqrt{\text{MSE}}$
MAE: $\frac{1}{N} \sum |y_i - \hat{y}_i|$ MAPE: $\frac{1}{N} \sum \left| \frac{y_i - \hat{y}_i}{y_i} \right|$

2 Logistic Regression

Binary Model

$$\hat{y}_i = \sigma(w^T x_i), \quad \nabla_w R(w) = \sum_i (\hat{y}_i - y_i) x_i$$
$$R(w) = - \sum_i [y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i)]$$

No closed form: use GD. **Boundary:** $w^T x = 0$.
 $\tau \uparrow \Rightarrow \uparrow \text{Prec}$; $\tau \downarrow \Rightarrow \uparrow \text{Rec}$.

Least Square vs Logistic vs SVM

The main diff. between them is the loss function.

Confusion Matrix		
	Pred Pos	Pred Neg
True Pos	TP	FN (type II)
True Neg	FP (type I)	TN

Metrics & When to Use		
Metric	Formula	Use when
Precision	$\frac{TP}{TP+FP}$	$\downarrow$ FP: $\uparrow \tau$ (spam)
Recall	$\frac{TP}{TP+FN}$	$\downarrow$ FN: $\downarrow \tau$ (cancer)
F1	$\frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$	balance FP&FN
Accuracy	$\frac{TP+TN}{N}$	misleading if imbalanced
FPR	$\frac{FP}{FP+TN}$	ROC x-axis

ROC: TPR= $f(\text{FPR})$ varying τ .
 $\Rightarrow$ AUC= 1: perfect, ≈ 0.5 : random.

Softmax Model ($C > 1$)

$$\hat{y}^{(k)}(x) = \frac{\exp(w^{(k)T} x)}{\sum_{j=1}^C \exp(w^{(j)T} x)}$$
$$R(W) = - \sum_i \sum_{k=1}^C y_i^{(k)} \ln \hat{y}_i^{(k)}$$
$$\nabla_W R = \sum_i x_i (\hat{y}_i - y_i)^T$$

3 Support Vector Machine (SVM)

When: binary classification; want maximum-margin boundary; works well in high dim.

Margin

Binary labels $y_i \in \{-1, +1\}$. Hyperplane $w^T x + b = 0$. Margin = $2/\|w\|$. Support vectors are the closest points defining the margin.

Hard-margin SVM

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i (w^T x_i + b) \geq 1 \quad \forall i$$

Use when data is linearly separable; no misclassification allowed.

Hinge Loss

$$\min_{w,b} \frac{1}{2C} \|w\|^2 + \sum_i \max(0, 1 - y_i (w^T x_i + b))$$

$C \uparrow$: smaller margin, more risk of overfit. $C \downarrow$: wider margin, more violations.
Prediction: $\hat{y} = \text{sign}(w^T x + b)$. L2 regularization.

4 k-Nearest Neighbors

No training beyond storing data !

Classification: majority vote.
Regression: average or weighted average
Normalize ; Small k : outlier-sensitive, overfit risk.
Large k : smoother, underfit risk.

5 Feature Expansion + CV

Why: when ϕ is very high-dim or infinite-dim, computing $\phi(x)$ is infeasible. We only need dot products $\phi(x_i)^T \phi(x_j) = k(x_i, x_j)$.

Over/Underfitting

Underfit: train and test errors $\uparrow$; model too simple.
Overfit: train error $\downarrow$, valid./test error $\uparrow$; complex.

Cross-validation

Never use test data to choose hyperparameters.
Validation set: fast, split-dependent.
LOOCV: train N times, expensive.
K-fold: train K times, common $K = 5, 10$.

Overfitting Risk

High-degree $\phi \Rightarrow F \gg N \Rightarrow$ overfitting.

6 Ridge + Kernels

Kernel vs explicit ϕ : kernel is $O(N^2)$ in space vs $O(NF)$

L2 / Ridge Regression

$$E(w) = \sum_i (\phi(x_i)^T w - y_i)^2 + \lambda \|w\|^2$$
$$w^* = (\Phi^T \Phi + \lambda I_F)^{-1} \Phi^T y$$

$\lambda \uparrow$: weights shrink toward 0, simpler model, underfit as $\lambda \rightarrow \infty$. Adding λI helps invertibility issue.

Kernel Trick

$$k(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$
$$K_{ij} = k(x_i, x_j) \in \mathbb{R}^{N \times N} \quad K = K^T, \quad K \succeq 0$$

Representer Theorem

The minimizer of any regularized empirical risk:

$$w^* = \Phi^T \alpha = \sum_i \alpha_i \phi(x_i)$$

Optimize α instead of $w \Rightarrow$ everything via K .

Kernel Ridge Regression

$$\alpha^* = (K + \lambda I_N)^{-1} y$$
$$\hat{y} = y^T (K + \lambda I_N)^{-1} k(X, x) \quad (\text{same with Y})$$

7 Data Representation

Normalization

$$\text{min-max: } x \leftarrow \frac{x - \min}{\max - \min} \in [0, 1]$$
$$\text{z-score: } x \leftarrow \frac{x - \mu}{\sigma}$$
$$\text{max: } x \leftarrow x / \max, \quad \text{decimal: } x \leftarrow x / 10^j$$

Images: Bag of Words (BoW)

$$\mathbf{x}_1 = [1/8 \quad 1/4 \quad 1/8 \quad 1/4 \quad 1/8 \quad 1/8 \quad 0 \quad 0 \quad 0]$$

Images: Visual BoW

Extract patches $\rightarrow$ cluster into D visual words $\rightarrow$ count $\rightarrow$ normalize $\Rightarrow x_i \in \mathbb{R}^D$.

Missing Data

Drop: loses info
Global mean: fast, ignores class
Class-conditioned mean: best

Noise / Binning

Sort $\rightarrow$ bins $\rightarrow$ replace by bin mean/median.

8 Imbalanced Data

Sampling Methods

Oversampling: + minority; naive duplication can overfit; SMOTE-like neighbors can add noise.
Undersampling: - majority; random removal may discard important samples; remove redundancy.

Cost-sensitive Risk

$$R = \frac{1}{N} \sum_i \beta_i \ell(\hat{y}_i, y_i)$$

Do not optimize β_i : trivial 0-loss solutions.

9 Neural Networks

Backpropagation

For neuron k in layer l :

$$a_{(l)}^{(k)} = \sum_j W_{(l)}^{(k,j)} z_{(l-1)}^{(j)}, \quad z_{(l)}^{(k)} = f(a_{(l)}^{(k)})$$
$$\delta_{(l)}^{(k)} = \frac{\partial \ell_i}{\partial a_{(l)}^{(k)}}, \quad \frac{\partial \ell_i}{\partial W_{(l)}^{(k,j)}} = \delta_{(l)}^{(k)} z_{(l-1)}^{(j)}$$

Mini-Batch SGD

$$W_k \leftarrow W_{k-1} - \eta \sum_{b=1}^B \nabla \ell_{(b,k)}(W_{k-1})$$

B = batch size. Full GD: $B = N$; SGD: $B = 1$; mini-batch: $1 < B < N$.

SGD with Momentum

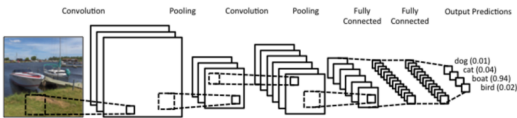
$$\nu \leftarrow \gamma \nu + \eta \nabla R(W), \quad W \leftarrow W - \nu$$

$\gamma > 0$ adds inertia; accelerates if ∇ is consistent.

Neural Network Regularization

Dropout: zero each neuron w.p. p during train.
Early stopping: stop when val. loss starts rising.
Weight decay: $+\lambda \|W\|_F^2$ in loss (L2 for NNs).
Data augmentation: flip/crop/rotate.

10 Convolutional Neural Network (CNN)



Output Size & Params

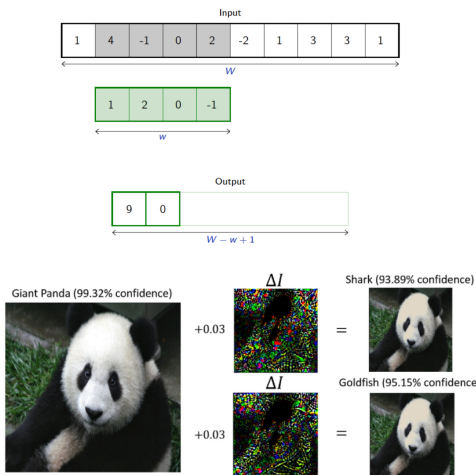
Output size: $\lfloor (W - F + 2P)/S \rfloor + 1$.
 Same padding: $P = (F - 1)/2$.
 Params: $(F^2 C_{\text{in}} + 1) \times C_{\text{out}}$ (far fewer than FC).

Transposed Conv

It upsamples: output size $(W-1)S + F$.

CNN Tricks

Convs: translation equivariance.
 Pooling/stride: downsampling + robustness.



11 Dimensionality Reduction

Limitation: only captures linear structure.

PCA Projection

$$\bar{x} = \frac{1}{N} \sum_i x_i, \quad y_i = W^T(x_i - \bar{x}), \quad W^T W = I_d$$

Covariance:

$$C = \frac{1}{N} \sum_i (x_i - \bar{x})(x_i - \bar{x})^T$$

1D objective:

$$\max_{\|w\|=1} w^T C w \Rightarrow Cw = \lambda w$$

PCA Reconstruction

$$\hat{x}_i = \bar{x} + W y_i = \bar{x} + W W^T (x_i - \bar{x})$$

PCA minimizes reconstruction error among linear orthogonal projections. (Uncorrelated = covariance matrix diagonal in the new space). Keep d such that:

$$\sum_{j=1}^d \lambda_j \geq 0.9 \sum_{k=1}^D \lambda_k$$

Reduces cost and can regularize

Invariant under addition ($x_i + c$): mean shifts by c , covariance unchanged $\Rightarrow$ same eigenvectors/values.

Not invariant under scaling (Dx_i): covariance changes $\Rightarrow$ eigenvectors change. Normalize first.

12 Kernel PCA + Autoencoders

When: non-linear structure in data.

Kernel PCA

$$x_i \mapsto \phi(x_i), \quad K_{ij} = k(x_i, x_j)$$

Representer:

$$w = \sum_{j=1}^N a_j \phi(x_j), \quad Ka = \lambda Na$$

Projection:

$$y_i = \phi(x_i)^T w = \sum_{j=1}^N a_j k(x_i, x_j)$$

Kernel PCA maps high $\rightarrow$ low, but unlike PCA it does not naturally provide low $\rightarrow$ high reconstruction.

When: non-linear structure; need encoder & decoder; large data.

Autoencoder

Encoder: input $\rightarrow$ latent representation. **Decoder:** latent $\rightarrow$ reconstruction. Train with:

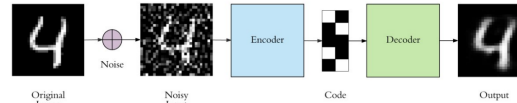
$$R = \frac{1}{N} \sum_i \|\hat{x}_i - x_i\|^2$$

Non-convex; trained by SGD/backprop. Decoder gives low/latent $\rightarrow$ output mapping, so latent vectors can generate samples.

Variants

Deep AE: stacked layers $z^{(j)} = f^{(j)}(W_{(j)}^T z^{(j-1)})$.

Denoising AE: corrupt input, reconstruct clean input; learns robust representation.



13 K-means

Complexity: $O(NKD)$ per iteration.

Problem + Algorithm

Unsupervised clustering into K groups with compact clusters.

1. Initialize centers $\mu_1, \dots, \mu_K$.
2. Assign each x_i to nearest μ_k .
3. Update each μ_k as mean of assigned points.
4. Repeat until assignments/centers stop changing.

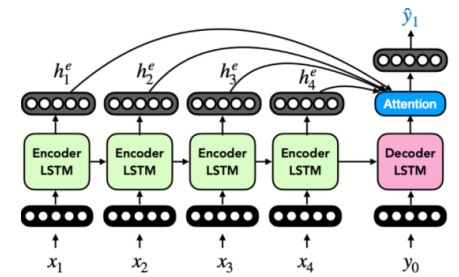
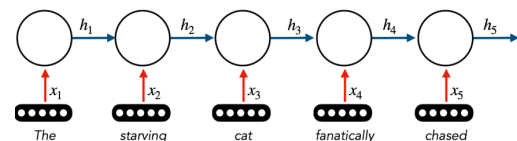
K-means Tricks

Initialization matters; run multiple initializations. Euclidean distance makes feature scaling crucial. Elbow method helps choose K . K-means exploits compactness; fails on non-compact clusters such as concentric circles.

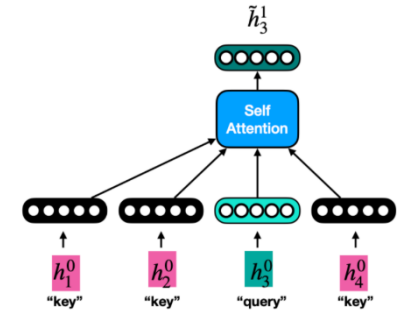
14 NLP, RNNs, Attention

Word Representations

Word2Vec: dense $\mathbb{R}^d$ embeddings trained self-supervised from context; similar words $\approx$ nearby vectors (king - man + woman $\approx$ queen).



15 Transformers



Transformer Blocks

Transformer encoder/decoder are stacks of blocks with multi-head attention, feed-forward layers, and Add & Norm. Main advantage over RNNs: self-attention can process inputs in parallel instead of sequentially.

Decoder Details

Masked self-attention prevents attending to future tokens. Cross-attention in the decoder attends over encoder outputs. GPT is a decoder transformer using masked multi-head self-attention and no cross-attention.

Common Checks

RBF kernel has an infinite-dimensional feature view. kNN works for regression and classification; k is not learned by GD.

K-means converges, but not always to the desired/global solution.

PCA components are ordered by eigenvalues; labels do not change PCA.

Parameter Counting

Fully connected layer $n_{\text{in}} \rightarrow n_{\text{out}}$:

$$\# \text{params} = (n_{\text{in}} + 1)n_{\text{out}}$$

Convolution layer with kernel $H_K \times W_K$, C_{in} input channels, C_{out} filters:

$$\# \text{params} = (H_K W_K C_{\text{in}} + 1)C_{\text{out}}$$